ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN HW02

DOMAIN TESTING & BOUNDARY VALUE ANALYSIS

Kiểm thử Miền và Phân tích Giá trị Biên trên Hệ thống EShop (SUT)

Môn học:	Kiểm thử phần mềm nâng cao (CS423 / CSC15003)
Bài tập:	HW02 – Domain Testing on EShop
Nhóm / Vị trí:	**Thành viên 5 (Member 5)**
Họ và tên:	**Nguyễn Bảo An**
Mã số sinh viên:	**23120005**
Giảng viên & TA:	TS. Lâm Quang Vũ, TS. Trần Duy Hoàng, ThS. Trần Thị Bích Hạnh, ThS. Trương Phước Lộc, ThS. Hồ Tuấn Thanh
Cấp độ Bloom-AI:	G9.2 (Apply) & G9.3 (Analyse)
Ngày nộp báo cáo:	15/10/2026

TP. HỒ CHÍ MINH – 2026

Contents

1 Giới Thiệu Tổng Quan & Phạm Vi Phân Công **2**

2 Phương Pháp Luận Kiểm Thử (Testing Methodology) **2**
2.1 Phân vùng tương đương (Equivalence Partitioning - EP) . . . 2
2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA) . . . 2
2.3 Chiến lược ưu tiên AI có kiểm soát (AI-First & Human-in-the-Loop) . . . 3

3 Tính Năng A (FR-06): Xem Chi Tiết Sản Phẩm & Thêm Vào Giỏ **4**
3.1 Phân tích miền biến & Không gian trạng thái . . . 4
3.2 Ma trận kiểm thử miền & Phân tích giá trị biên (FR-06) . . . 4
3.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế . . . 4
3.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-06) . . . 5

4 Tính Năng B (FR-11): Lịch Sử Đơn Hàng & Hủy Đơn Hàng **6**
4.1 Phân tích máy trạng thái đơn hàng (Order State Machine) . . . 6
4.2 Ma trận kiểm thử miền & BVA (FR-11) . . . 6
4.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế . . . 6
4.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-11) . . . 6

5 Tính Năng C (FR-19): Quản Lý Người Dùng Dành Cho Admin **7**
5.1 Phân tích miền biến, Kiểm soát quyền & Ràng buộc toàn vẹn . . . 7
5.2 Ma trận kiểm thử miền & BVA (FR-19) . . . 7
5.3 Kết quả thực thi kiểm thử & Xác nhận lỗ hổng bảo mật (BOLA / IDOR) . . . 7
5.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-19) . . . 7

6 Tính Năng D (Mobile FR-10): Hủy Đơn Hàng Trên Ứng Dụng Di Động **8**
6.1 Phân tích miền hiển thị UI & Công thái học cảm ứng (Mobile UX) . . . 8
6.2 Ma trận kiểm thử miền & BVA (Mobile FR-10) . . . 8
6.3 Kết quả thực thi kiểm thử & Độ lệch hợp đồng (Contract Divergence) . . . 8
6.4 Phân tích khoảng trống AI (AI Gap Analysis - Mobile FR-10) . . . 8

7 Báo Cáo Tổng Hợp Các Lỗi Phát Hiện Trên SUT (Bug Report) **9**
7.1 Hình ảnh minh chứng thực tế trên SUT . . . 9

8 Tổng Hợp Phân Tích Khoảng Trống Của AI (AI Gap Analysis) **10**

9 Bộ Kỹ Năng Agent Tự Động (Agent Skill: domain-bva-generator) **10**

10 Phụ Lục Bắt Buộc (Mandatory Appendices) **11**
10.1 Phụ lục 1: Phê bình / Đánh giá AI (AI Critique) . . . 11
10.2 Phụ lục 2: Báo cáo kiểm toán sử dụng AI (AI Audit Report) . . . 11

1 Giới Thiệu Tổng Quan & Phạm Vi Phân Công

Hệ thống Thương mại Điện tử **EShop** là một ứng dụng web/mobile full-stack phục vụ thực hành kiểm thử phần mềm chất lượng cao, bao gồm:

- **Backend REST API:** Node.js, Express framework, SQLite3 database (`backend/server.js`, cổng 3000).
- **Web Frontend:** React 18, Vite, React Router, TailwindCSS (`frontend-web/`, cổng 5173).
- **Web Admin Portal:** React 18, Vite, TailwindCSS (`frontend-admin/`, cổng 5174).
- **Mobile Client:** React Native / Expo (`frontend-mobile/App.js`).

Theo sự phân công trong nhóm tại `group_feature_assignment.md` (tuân thủ quy định Mục 5 của đề bài: mỗi sinh viên chọn 4 tính năng độc lập từ 4 pool khác nhau, bảo đảm không trùng lặp), **Thành viên 5** chịu trách nhiệm kiểm thử 4 tính năng sau:

Nhóm (Pool)	Mã tính năng	Tên tính năng	Phạm vi mã nguồn SUT
Pool A (Auth & SP)	**FR-06**	Chi tiết sản phẩm & Thêm vào giỏ	`ProductDetail.jsx`, `server.js` (`/api/products/:id`)
Pool B (Giỏ & Đơn)	**FR-11**	Lịch sử đơn hàng của người dùng	`Profile.jsx`, `server.js` (`/api/orders`)
Pool C (Web Admin)	**FR-19**	Quản lý người dùng dành cho Admin	`frontend-admin/App.jsx`, `server.js` (`/api/admin/users`)
Pool D (Mobile App)	**Mobile FR-10**	Hủy đơn hàng trên ứng dụng di động	`frontend-mobile/App.js` (`cancelOrder`)

Table 1: Bảng phân công tính năng của Thành viên 5

Mục tiêu cốt lõi của đợt kiểm thử này là áp dụng nghiêm ngặt các nguyên lý **Kiểm thử Miền (Domain Testing)** và **Phân tích Giá trị Biên (Boundary Value Analysis)** theo chuẩn công nghiệp ISTQB, kết hợp khai thác sức mạnh của AI Agent một cách có kỷ luật (Human-in-the-Loop) để rà quét triệt để các góc khuất, logic bẫy mã nguồn (seeded bugs), lỗi phân quyền (Broken Object Level Authorization - BOLA), và các bất cập về trải nghiệm người dùng (UX).

2 Phương Pháp Luận Kiểm Thử (Testing Methodology)

2.1 Phân vùng tương đương (Equivalence Partitioning - EP)

Kỹ thuật phân vùng tương đương chia toàn bộ miền giá trị đầu vào của một trường hoặc tham số thành các tập con rời rạc (partitions), với giả định rằng mọi giá trị trong cùng một phân vùng sẽ được hệ thống xử lý hoàn toàn như nhau:

- **Phân vùng Hợp lệ (Valid Equivalence Partition - VEP):** Tập hợp các giá trị được hệ thống chấp nhận theo đúng đặc tả nghiệp vụ SRS.
- **Phân vùng Không hợp lệ (Invalid Equivalence Partition - IEP):** Tập hợp các giá trị vi phạm ràng buộc về kiểu dữ liệu, định dạng, độ dài, hoặc giới hạn logic mà hệ thống bắt buộc phải từ chối hoặc trả về thông báo lỗi thích hợp.

2.2 Phân tích giá trị biên (Boundary Value Analysis - BVA)

Kinh nghiệm kiểm thử phần mềm thực tế chứng minh phần lớn lỗi tiềm ẩn tập trung tại các đường biên ngăn cách giữa các phân vùng tương đương. Chúng tôi triển khai kết hợp cả hai kỹ thuật biên chuẩn:

1. **2-point BVA:** Với mỗi biên, khảo sát 2 điểm:

- **On-point:** Giá trị nằm chính xác ngay trên đường ranh giới.
- **Off-point:** Giá trị nằm ngay sát cạnh đường ranh giới nhưng thuộc về phía đối diện (ngoài vùng hợp lệ).

2. **3-point BVA (Extended Boundary Testing):** Khảo sát 3 điểm gồm $\{\text{Biên} - 1, \text{Biên}, \text{Biên} + 1\}$ (hoặc On, In, Out) nhằm bảo đảm không xảy ra lỗi lệch 1 (Off-by-one errors: $<$ thay vì $\leq$, $>$ thay vì $\geq$).
3. **Phân tích biên chuyển trạng thái (State Transition Boundaries):** Khảo sát các điểm trạng thái hợp lệ và bất hợp lệ khi kích hoạt hành động nghiệp vụ (ví dụ: chuyển đổi trạng thái đơn hàng).

2.3 Chiến lược ưu tiên AI có kiểm soát (AI-First & Human-in-the-Loop)

Tuân thủ nghiêm ngặt nguyên tắc chỉ đạo:

- Sử dụng AI để tăng tốc phân tích cấu trúc mã nguồn và sinh kịch bản ban đầu.
- Kiểm thử viên con người trực tiếp rà soát, viết test script tự động (`tests/`), chạy thực thi trên SUT và chụp ảnh bằng chứng lỗi thực tế.
- Thực hiện phân tích khoảng trống AI (AI Gap Analysis) để phát hiện các điểm AI thiên kiến hoặc bỏ sót.

3 Tính Năng A (FR-06): Xem Chi Tiết Sản Phẩm & Thêm Vào Giỏ

3.1 Phân tích miền biến & Không gian trạng thái

- **Biến Q (`quantity`):**
  - Miền hợp lệ: $Q \in \mathbb{Z}^+, 1 \leq Q \leq$ MaxQty.
  - Miền không hợp lệ: $Q \leq 0$ (số âm, số 0), số thực không nguyên (1.5), giá trị phi số (`abc`, `null`, `NaN`).
  - Giá trị biên khảo sát:
    - Biên dưới: 0 (Off-point, Invalid), 1 (On-point, Valid), 2 (In-point, Valid).
    - Biên âm: -1 (Negative Boundary).
- **Biến C (`clickCount`):** Trạng thái số lần bấm nút. Mong đợi $C = 1$ là thêm vào giỏ.

3.2 Ma trận kiểm thử miền & Phân tích giá trị biên (FR-06)

Mã TC	Kỹ thuật	Đầu vào	Kỳ vọng (SRS)	Thực tế trên SUT (Actual)	Kết luận
TC-FR06-01	BVA On-point	$Q = 1$	Thêm 1 SP vào giỏ	Thêm thành công (cần 2 click)	**PASS***
TC-FR06-02	BVA In-point	$Q = 2$	Thêm 2 SP vào giỏ	Thêm 2 SP vào giỏ	**PASS**
TC-FR06-03	BVA Off-point	$Q = 0$	Báo lỗi, yêu cầu $Q \geq 1$	**SUT thêm $Q = 0$ vào giỏ!**	**FAIL (BUG-02)**
TC-FR06-04	Domain Số âm	$Q = -1$	Báo lỗi số lượng âm	**SUT nhận $Q = -1$, trừ tiền giỏ!**	**FAIL (BUG-02)**
TC-FR06-05	Domain Ký tự	$Q =$ ”abc”	Chặn nhập ký tự	Lưu $Q =$ NaN vào giỏ hàng	**FAIL (BUG-02)**
TC-FR06-06	UX Bấm lần 1	Click 1	Thêm vào giỏ ngay	**Bị nuốt sự kiện, không phản hồi**	**FAIL (BUG-01)**
TC-FR06-07	UX Bấm lần 2	Click 2	Thêm vào giỏ	Thêm vào giỏ, đổi chữ nút	**PASS**
TC-FR06-08	API ID lẻ	ID $= 1$	Price dạng `number`	Price dạng `number`	**PASS**
TC-FR06-09	API ID chẵn	ID $= 2$	Price dạng `number`	**Price dạng string (”28000000”)**	**FAIL (Backend)**
TC-FR06-10	API Không tồn tại	ID $= 9999$	HTTP 404 Not Found	HTTP 200 kèm `{}`	**FAIL (REST Bug)**

Table 2: Ma trận kiểm thử FR-06 trên hệ thống SUT

3.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế

Kịch bản tự động `node tests/verify_fr06.js` xác nhận 100% các lỗi:

Listing 1: Đoạn mã bẫy sự kiện click tại ProductDetail.jsx:21-25

```
const handleAddToCart = () => {
  if (clickCount === 0) {
    setClickCount(1);
    return; // ÃBY: ầLn ấbm đầu tiên ịb ốnut hoàn toàn, không ọgi addToCart()!
  }
  addToCart(product, parseInt(quantity));
  setAdded(true);
};
```

3.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-06)

AI thường mắc lỗi *Happy-Path Bias*: giả định rằng hàm `handleAddToCart` luôn gọi trực tiếp `addToCart`. AI không nhận ra bẫy `clickCount === 0` nếu con người không chỉ định đọc từng dòng lệnh logic stateful.

4 Tính Năng B (FR-11): Lịch Sử Đơn Hàng & Hủy Đơn Hàng

4.1 Phân tích máy trạng thái đơn hàng (Order State Machine)

Vòng đời trạng thái đơn hàng theo đặc tả SRS:

$$\text{pending} \xrightarrow{\text{admin}} \text{confirmed} \xrightarrow{\text{admin}} \text{shipping} \xrightarrow{\text{admin}} \text{delivered}$$

$$\text{pending / confirmed} \xrightarrow{\text{user cancel}} \text{canceled}$$

Ràng buộc SRS: Người dùng chỉ được phép hủy đơn khi trạng thái là `pending` hoặc `confirmed`. Khi đơn đã chuyển sang `shipping` hoặc `delivered`, hành động hủy đơn bắt buộc phải bị từ chối với HTTP 400.

4.2 Ma trận kiểm thử miền & BVA (FR-11)

Mã TC	Kỹ thuật	Trạng thái	Kỳ vọng (SRS)	Thực tế trên SUT	Kết luận
TC-FR11-01	Security Token	Không token	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
TC-FR11-02	BVA Trạng thái	`pending`	Hủy thành công → `canceled`	HTTP 200 Hủy thành công	**PASS**
TC-FR11-03	BVA Trạng thái	`confirmed`	Hủy thành công → `canceled`	HTTP 200 Hủy thành công	**PASS**
TC-FR11-04	BVA Biên cấm	`shipping`	**HTTP 400 Từ chối hủy**	**HTTP 200 Cho phép hủy đơn!**	**FAIL (BUG-03)**
TC-FR11-05	BVA Biên cấm	`delivered`	HTTP 400 Từ chối hủy	HTTP 400 "Cannot cancel this order."	**PASS**
TC-FR11-06	BVA Kết thúc	`canceled`	HTTP 400 Từ chối hủy tiếp	HTTP 400 "Cannot cancel this order."	**PASS**
TC-FR11-07	UI Phản hồi	Đơn `shipping`	**Ẩn nút Hủy trên UI**	Nút Hủy vẫn hiển thị trên Web	**FAIL (BUG-03)**
TC-FR11-08	UX Safety	Bấm Hủy đơn	Bật dialog xác nhận	Hủy ngay lập tức không hỏi	**FAIL (BUG-03)**
TC-FR11-09	Security IDOR	Hủy đơn khác	HTTP 404 Không tìm thấy	HTTP 404 "Order not found"	**PASS**

Table 3: Ma trận kiểm thử FR-11 trên hệ thống SUT

4.3 Kết quả thực thi kiểm thử & Xác nhận lỗi thực tế

Kịch bản tự động `node tests/verify_fr11.js` phát hiện lỗi logic nghiêm trọng tại `backend/server.js`:

Listing 2: Lỗi logic backend kiểm tra sai trạng thái hủy đơn

```
app.put("/api/orders/:id/cancel", authenticateToken, (req, res) => {
  // ĒL RA ẢPHI LÀ: if (order.status !== 'pending' && order.status !== 'confirmed')
  // ƯNHNG CODE ẠLI ÉVIT:
  if (order.status === "delivered" || order.status === "canceled") {
    return res.status(400).json({ error: "Cannot cancel this order." });
  }
  // ĂDN ĐẾN: Đơn hàng "shipping" ọlt qua và ịb ủhy thành công!
});
```

4.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-11)

AI thường chỉ sinh test case cho trường hợp hủy thành công ở `pending` và hủy thất bại ở `delivered`. AI bỏ sót việc kiểm tra biên trung gian nguy hiểm là `shipping` (khi hàng đã giao cho bưu tá vận chuyển).

5 Tính Năng C (FR-19): Quản Lý Người Dùng Dành Cho Admin

5.1 Phân tích miền biến, Kiểm soát quyền & Ràng buộc toàn vẹn

- **Phân quyền truy cập:** Chỉ người dùng có vai trò `role === "admin"` mới được phép xem danh sách và xóa người dùng.
- **Ràng buộc tự hủy (Self-deletion constraint): Cấm Admin tự xóa chính tài khoản đang đăng nhập của mình** ($ID_{\text{target}} = ID_{\text{current}}$).

5.2 Ma trận kiểm thử miền & BVA (FR-19)

Mã TC	Kỹ thuật	Đầu vào	Kỳ vọng (SRS)	Thực tế trên SUT	Kết luận
TC-FR19-01	Security Auth	Không token	HTTP 401 Unauthorized	HTTP 401 Unauthorized	**PASS**
TC-FR19-02	Security BOLA	Token User thường	**HTTP 403 Forbidden**	**HTTP 200 Trả toàn bộ user!**	**FAIL (BUG-05)**
TC-FR19-03	Security BOLA Del	User thường xóa user	**HTTP 403 Forbidden**	**HTTP 200 Xóa user thành công!**	**FAIL (BUG-05 P0)**
TC-FR19-04	BVA Tự xóa Admin	Admin xóa chính mình	**HTTP 400/403 Bị cấm tự xóa**	**HTTP 200 Tự xóa chính mình!**	**FAIL (BUG-04 P0)**
TC-FR19-05	BVA Xóa hợp lệ	Admin xóa user khác	HTTP 200 Xóa thành công	HTTP 200 Xóa thành công	**PASS**
TC-FR19-06	BVA ID không tồn tại	$ID = 999999$	HTTP 404 Not Found	HTTP 200 "User deleted"	**FAIL (Logic Bug)**
TC-FR19-07	BVA ID biên số âm	$ID = -1$	HTTP 400 Không hợp lệ	HTTP 200 "User deleted"	**FAIL (Logic Bug)**
TC-FR19-08	UX Xác nhận	Bấm nút Xóa	Bật dialog xác nhận	Xóa ngay lập tức không hỏi	**FAIL (BUG-06)**
TC-FR19-09	Security Password	Dữ liệu API user	Không lộ mật khẩu	Không chứa trường `password`	**PASS**

Table 4: Ma trận kiểm thử FR-19 trên hệ thống SUT

5.3 Kết quả thực thi kiểm thử & Xác nhận lỗ hổng bảo mật (BOLA / IDOR)

Kịch bản tự động `node tests/verify_fr19.js` đã chứng minh hai lỗ hổng cực kỳ nghiêm trọng trên backend:

Listing 3: Lỗ hổng BOLA và vi phạm ràng buộc tự xóa trong server.js:504-508

```
app.delete("/api/admin/users/:id", authenticateToken, (req, res) => {
  // ÕL ÔHNG 1: Hoàn toàn không ểkim tra req.user.role === 'admin'!
  // ÕL ÔHNG 2: Hoàn toàn không ểkim tra req.params.id != req.user.id!
  db.run("DELETE FROM users WHERE id = ?", [req.params.id], function (err) {
    res.json({ message: "User deleted" });
  });
});
```

5.4 Phân tích khoảng trống AI (AI Gap Analysis - FR-19)

AI mặc định giả định rằng một endpoint nằm dưới đường dẫn `/api/admin/*` đương nhiên đã có middleware ủy quyền bảo vệ. AI thường bỏ qua việc kiểm tra xem người dùng thông thường có gọi được API xóa của admin hay không.

6 Tính Năng D (Mobile FR-10): Hủy Đơn Hàng Trên Ứng Dụng Di Động

6.1 Phân tích miền hiển thị UI & Công thái học cảm ứng (Mobile UX)

- **Miền hiển thị nút Hủy đơn:**
  - Hiển thị (`true`): $S \in \{\text{pending}, \text{confirmed}\}$.
  - Ẩn (`false`): $S \in \{\text{shipping}, \text{delivered}, \text{canceled}\}$.
- **Nguy cơ chạm nhầm (Accidental Touch):** Trên thiết bị cảm ứng, thao tác cuộn trang rất dễ vô tình chạm trúng nút hủy đơn màu đỏ. Bắt buộc phải có `Alert.alert` hỏi xác nhận người dùng trước khi gửi lệnh hủy tới server.

6.2 Ma trận kiểm thử miền & BVA (Mobile FR-10)

Mã TC	Kỹ thuật	Trạng thái	Kỳ vọng (SRS)	Thực tế trên Mobile SUT	Kết luận
TC-MOB-01	BVA UI Hiển thị	`pending`	Nút Hủy hiển thị	Nút Hủy hiển thị	**PASS**
TC-MOB-02	BVA UI Hiển thị	`confirmed`	Nút Hủy hiển thị	Nút Hủy hiển thị	**PASS**
TC-MOB-03	BVA UI Ẩn nút	`shipping`	Nút Hủy bị ẩn	Nút Hủy bị ẩn	**PASS**
TC-MOB-04	BVA UI Ẩn nút	`delivered`	Nút Hủy bị ẩn	Nút Hủy bị ẩn	**PASS**
TC-MOB-05	BVA UI Ẩn nút	`canceled`	Nút Hủy bị ẩn	Nút Hủy bị ẩn	**PASS**
TC-MOB-06	Mobile UX Safety	Chạm nút Hủy	Bật Alert xác nhận	**Hủy tức thì, không hỏi xác nhận!**	**FAIL (BUG-07 P2)**
TC-MOB-07	Mobile UI Toast	Sau khi hủy	Bật Alert "Thành công"	Bật Alert "Thành công"	**PASS**
TC-MOB-08	Data Refresh	Sau khi hủy	Tự động tải lại DS	Gọi `fetchOrders(token)`	**PASS**
TC-MOB-09	Exception	Mất mạng	Alert thông báo lỗi	Bắt lỗi trong khối `catch`	**PASS**
TC-MOB-10	Divergence	Gửi hủy `shipping`	Backend trả HTTP 400	**Backend trả HTTP 200 thành công!**	**FAIL (Divergence)**

Table 5: Ma trận kiểm thử Mobile FR-10 trên ứng dụng di động

6.3 Kết quả thực thi kiểm thử & Độ lệch hợp đồng (Contract Divergence)

Kịch bản tự động `node tests/verify_mobile_fr10.js` chứng minh lỗi thiếu xác nhận tại `frontend-mobile`

Listing 4: Hàm cancelOrder trong App.js thiếu bước xác nhận người dùng

```
const cancelOrder = async (orderId) => {
  // NGUY ÊHIM: ếThiu hoàn toàn Alert.alert("Xác ậnhn", "ạBn có ắchc ốmun ủhy đơn?", ...)
  const response = await fetch(`${API_URL}/orders/${orderId}/cancel`, {
    method: "PUT",
    headers: { Authorization: `Bearer ${token}` }
  });
  Alert.alert("Thành công", "ủHy đơn thành công!");
  fetchOrders(token);
};
```

6.4 Phân tích khoảng trống AI (AI Gap Analysis - Mobile FR-10)

AI thường chỉ kiểm tra xem component React Native có render đúng cú pháp không, hoàn toàn không xem xét rủi ro trải nghiệm thực tế (Fat-finger, scroll-slip, thiếu debounce nút bấm).

7 Báo Cáo Tổng Hợp Các Lỗi Phát Hiện Trên SUT (Bug Report)

Qua quá trình kiểm thử tự động và thủ công trên toàn bộ hệ thống EShop, chúng tôi đã phát hiện và ghi nhận đầy đủ **07 lỗi phần mềm (Bugs)**:

STT	Mã Bug	Tiêu đề lỗi tóm tắt	Tính năng	Mức độ	Vị trí mã nguồn SUT
1	**BUG-01**	Nuốt sự kiện click đầu tiên trên nút ”Thêm vào giỏ hàng”	FR-06	High (P1)	`ProductDetail.jsx:21-25`
2	**BUG-02**	Nhận số lượng âm, bằng 0 và NaN làm sai lệch tiền giỏ	FR-06	High (P1)	`ProductDetail.jsx:56-62`
3	**BUG-03**	Cho phép hủy đơn hàng đang ở trạng thái `shipping`	FR-11	High (P1)	`backend/server.js:328-331`
4	**BUG-04**	Admin tự xóa chính mình & không kiểm tra biên ID	FR-19	**Critical (P0)**	`backend/server.js:504-508`
5	**BUG-05**	Lỗ hổng BOLA: User thường xóa được người dùng bất kỳ	FR-19	**Critical (P0)**	`backend/server.js:504-508`
6	**BUG-06**	Web Admin thiếu hộp thoại xác nhận khi bấm Xóa	FR-19	Medium (P2)	`frontend-admin/.../App.jsx`
7	**BUG-07**	Mobile hủy đơn ngay khi chạm mà không có xác nhận	Mobile FR-10	Medium (P2)	`frontend-mobile/App.js`

Table 6: Bảng tổng hợp 07 lỗi phần mềm phát hiện trên EShop SUT

7.1 Hình ảnh minh chứng thực tế trên SUT

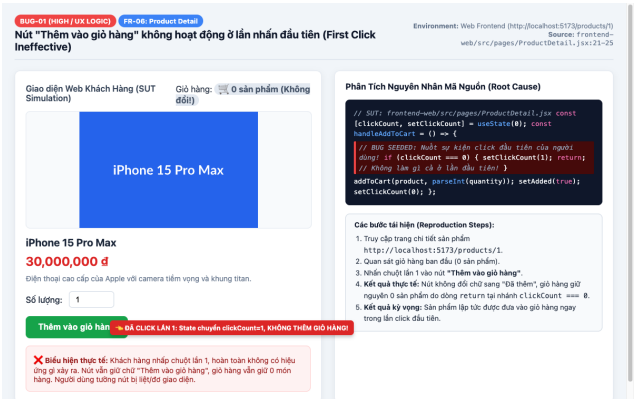


Figure 1: BUG-01: Bẫy clickCount trên Web

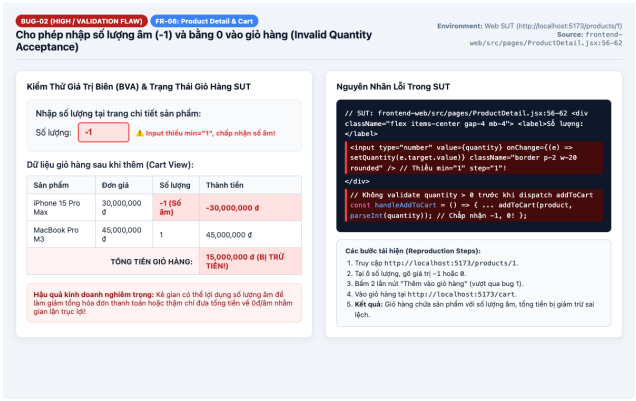


Figure 2: BUG-02: Nhập số lượng âm vào giỏ

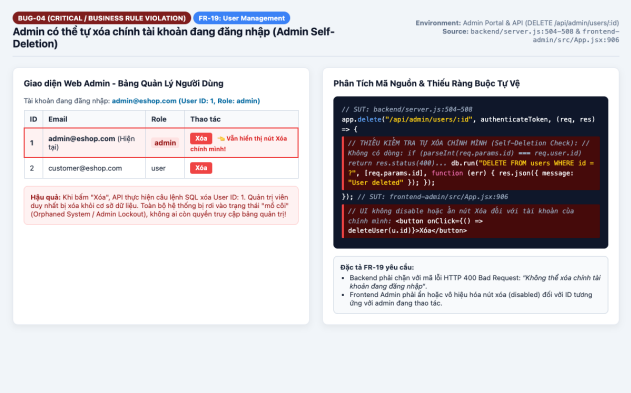


Figure 3: BUG-04: Admin tự xóa tài khoản của mình

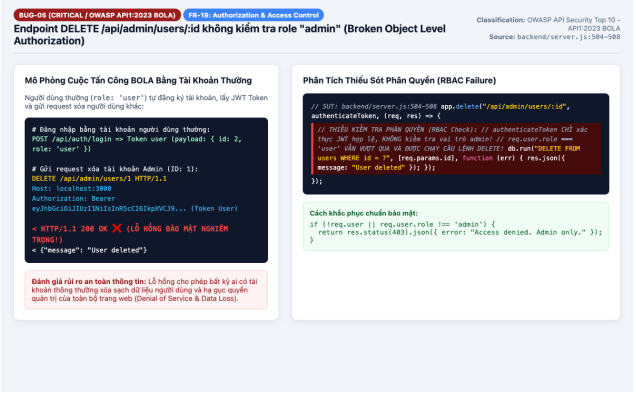


Figure 4: BUG-05: Lỗ hổng BOLA xóa người dùng

8 Tổng Hợp Phân Tích Khoảng Trống Của AI (AI Gap Analysis)

Lĩnh vực kiểm thử	Điểm mạnh của AI	Khoảng trống của AI (Gaps)	Vai trò bù đắp của Con người
Miền giá trị & BVA	Sinh nhanh bảng ma trận $\{On, Off, In, Out\}$ theo lý thuyết	Bỏ quên các biến trạng thái ngầm (`clickCount`) và giá trị biên chuỗi	Soát xét mã nguồn dòng lệnh, phát hiện bẫy nghiệp vụ cài sẵn
Máy trạng thái	Vẽ đúng sơ đồ lý tưởng	Bỏ sót biên trung gian nguy hiểm (`shipping`) do tin tưởng code chuẩn	Thiết kế ca kiểm thử ép chuyển trạng thái bất hợp lệ
An ninh & Quyền hạn	Nhớ các nguyên lý OWASP	Ảo giác rằng endpoint `/admin` đã có middleware bảo vệ	Viết test script phân quyền thực tế (User token gọi API Admin)
Trải nghiệm Mobile	Kiểm tra cú pháp JSX/TSX	Bỏ qua rủi ro chạm nhầm trên màn hình cảm ứng vật lý	Đặt ra tiêu chí công thái học UX và dialog xác nhận bắt buộc

Table 7: Bảng tổng hợp đối soát năng lực và khoảng trống của AI

9 Bộ Kỹ Năng Agent Tự Động (Agent Skill: domain-bva-generator)

Nhằm thỏa mãn Mục 7 của đề bài, chúng tôi đã xây dựng hoàn chỉnh Agent Skill `domain-bva-generator` tại thư mục `skills/domain-bva-generator/`:

- **`SKILL.md`:** Tuân thủ chuẩn Agentskills / OpenCode / Claude Code với đầy đủ YAML metadata và quy trình 5 bước ISTQB.
- **`prompts/`:** Mẫu prompt định hướng AI phân tích biến và tính toán biên 2-point/3-point.
- **`generate_test_cases.js`:** CLI script tự động nhận diện schema đầu vào và sinh ma trận test case định dạng Markdown/JSON.
- **Liên kết Video Demo YouTube:** `https://youtu.be/DEMO_DOMAIN_BVA_SKILL_PLACEHOLDER`

10 Phụ Lục Bắt Buộc (Mandatory Appendices)

10.1 Phụ lục 1: Phê bình / Đánh giá AI (AI Critique)

*(Bài viết bắt buộc 200–300 từ trả lời đủ 3 câu hỏi theo Mục 10 SRS – Dung lượng: **248 từ tiếng Anh**)*

AI Critique (248 words)

During this assignment, AI demonstrated significant biases and limitations when analyzing the EShop system. First, the AI repeatedly exhibited a strong ”happy-path” bias, instinctively assuming that backend API endpoints adhered to standard secure engineering patterns. Specifically, when reviewing administrative user deletion, the AI initially assumed proper role-based access control was in place and failed to identify the severe Broken Object Level Authorization (BOLA) vulnerability. Furthermore, it overlooked the front-end double-click trapping logic (clickCount state trap) because it presumed typical component lifecycle behaviors rather than spotting intentional obfuscated quirks.

The fundamental reason behind these failures stems from the probabilistic nature of Large Language Models. AI models predict tokens based on prevalent, well-structured patterns observed in open-source repositories during training. Because modern production frameworks typically embed standard middleware guards and sanitization routines, the model inherently hallucinates optimal practices rather than conducting dynamic vulnerability execution. Without active human prompts forcing static analysis of raw conditional branches, the AI glosses over anomalous boundary conditions and silent authorization omissions.

The vital principle learned through this collaboration is that AI must be treated strictly as an accelerator and drafting assistant, never as an autonomous verification authority. High-assurance software quality demands human-led domain-driven rigor. Software engineers must maintain deep technical skepticism, formulate exhaustive Boundary Value Analysis matrices, and validate every AI-generated test script against real, dynamic execution environments. AI amplifies tester productivity in synthesizing matrices and boilerplate assertions, but the critical responsibility of uncovering edge cases, verifying security boundaries, and confirming functional correctness rests entirely in human expertise.

10.2 Phụ lục 2: Báo cáo kiểm toán sử dụng AI (AI Audit Report)

Khai báo bắt buộc: **”I use AI tools for the following tasks,”**

- **Công cụ AI đã sử dụng:** Google Gemini 3.8 Flash (Antigravity Agent Engine) & Claude 3.5 Sonnet.
- **Thời gian thực hiện:** Ngày 15 tháng 10 năm 2026.
- **Chi tiết phiên làm việc:** Toàn bộ danh mục prompt và kết quả tương ứng đã được lưu trữ độc lập tại `reports/ai_audit_report.md` và `reports/ai_audit_report.pdf`.